

This website utilizes technologies such as cookies to enable essential site functionality, as well as for analytics, personalization, and targeted advertising. [Privacy Policy](#).

**Accept**

**Deny Non-Essential**

**Manage Preferences**

An attacker uses standard SQL injection methods to inject data into the command line for execution. This could be done directly through misuse of directives such as MSSQL\_xp\_cmdshell or indirectly through injection of data into the database that would be interpreted as shell commands. Sometime later, an unscrupulous backend application (or could be part of the functionality of the same application) fetches the injected data stored in the database and uses this data as command line arguments without performing proper validation. The malicious data escapes that data plane by spawning new commands to be executed on the host.

▼ **Likelihood Of Attack**

Low

▼ **Typical Severity**

Very High

▼ **Relationships**

|  | Nature    | Type                                                                                | ID  | Name                                                           |
|------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------|-----|----------------------------------------------------------------|
|                                                                                    | ChildOf   |  | 66  | <a href="#">SQL Injection</a>                                  |
|                                                                                    | CanFollow |  | 110 | <a href="#">SQL Injection through SOAP Parameter Tampering</a> |

|  | View Name                            | Top Level Categories                    |
|------------------------------------------------------------------------------------|--------------------------------------|-----------------------------------------|
|                                                                                    | <a href="#">Domains of Attack</a>    | <a href="#">Software</a>                |
|                                                                                    | <a href="#">Mechanisms of Attack</a> | <a href="#">Inject Unexpected Items</a> |

▼ **Execution Flow**

**Explore**

**Probe for SQL Injection vulnerability:** The attacker injects SQL syntax into user-controllable data inputs to search unfiltered execution of the SQL syntax in a query.

**Exploit**

- Achieve arbitrary command execution through SQL Injection with the MSSQL\_xp\_cmdshell directive:** The attacker leverages a SQL Injection attack to inject shell code to be executed by leveraging the xp\_cmdshell directive.
- Inject malicious data in the database:** Leverage SQL injection to inject data in the database that could later be used to achieve command injection if ever used as a command line argument
- Trigger command line execution with injected arguments:** The attacker causes execution of command line functionality which leverages previously injected database content as arguments.

▼ **Prerequisites**

The application does not properly validate data before storing in the database  
Backend application implicitly trusts the data stored in the database

None: No specialized resources are required to execute this type of attack.

#### Consequences



| Scope                                              | Impact                        | Likelihood |
|----------------------------------------------------|-------------------------------|------------|
| Integrity                                          | Modify Data                   |            |
| Confidentiality                                    | Read Data                     |            |
| Availability                                       | Unreliable Execution          |            |
| Confidentiality<br>Access Control<br>Authorization | Gain Privileges               |            |
| Confidentiality<br>Integrity<br>Availability       | Execute Unauthorized Commands |            |

#### Mitigations

Disable MSSQL xp\_cmdshell directive on the database

Properly validate the data (syntactically and semantically) before writing it to the database.

Do not implicitly trust the data stored in the database. Re-validate it prior to usage to make sure that it is safe to use in a given context (e.g. as a command line argument).

#### Example Instances

SQL injection vulnerability in Cacti 0.8.6i and earlier, when register\_argc\_argv is enabled, allows remote attackers to execute arbitrary SQL commands via the (1) second or (2) third arguments to cmd.php. NOTE: this issue can be leveraged to execute arbitrary commands since the SQL query results are later used in the polling\_items array and popen function ([CVE-2006-6799](#)).

Reference: <https://www.cve.org/CVERecord?id=CVE-2006-6799>

#### Related Weaknesses



| CWE-ID              | Weakness Name                                                                                      |
|---------------------|----------------------------------------------------------------------------------------------------|
| <a href="#">89</a>  | Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')               |
| <a href="#">74</a>  | Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection') |
| <a href="#">20</a>  | Improper Input Validation                                                                          |
| <a href="#">78</a>  | Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')         |
| <a href="#">114</a> | Process Control                                                                                    |

#### Content History

This website utilizes technologies such as cookies to enable essential site functionality, as well as for analytics, personalization, and targeted advertising. [Privacy Policy](#).